

Objetivos

Desenvolver uma aplicação em Python, com interface gráfica ou web, capaz de processar documentos em múltiplos formatos (PDF, DOCX e TXT), implementando uma pipeline de pré-processamento que permita extrair, limpar, estruturar e segmentar texto de forma consistente. A aplicação deverá preparar o conteúdo textual para utilização por modelos de linguagem de pequena e média dimensão (SLMs), assegurando a melhoria da qualidade do input através da normalização textual e da geração automática de prompts adequados.

Pretende-se ainda avaliar o impacto das diferentes etapas da pipeline na qualidade do texto final produzido pelos modelos, promovendo uma abordagem crítica e fundamentada sobre o processo de preparação de dados para sistemas de Inteligência Artificial..

Tarefas – etapas

1. Extração de Texto Multi-formato:

- I. Implementar mecanismos de upload de ficheiros;
- II. Suportar os formatos PDF, DOCX e TXT;
- III. Extração do texto bruto, preservando:
 - a. Quebras de linha originais;
 - b. Artefactos textuais (caracteres estranhos, símbolos inválidos, erros de *encoding*);
 - c. Inconsistências típicas de documentos digitalizados.
- IV. O texto extraído deve ser apresentado ao utilizador antes de qualquer transformação.

2. Pipeline de Limpeza e Pré-Processamento:

- I. Implementar uma pipeline configurável que prepare o texto para normalização. Esta pipeline pode incluir:
 - a. Remoção de artefactos;
 - b. Reconstrução de parágrafos;
 - c. Remoção de quebras de linha incorretas;
 - d. Normalização de espaços, pontuação e caracteres especiais;
 - e. Detecção e eliminação de cabeçalhos e rodapés repetidos.
- II. A aplicação deve permitir visualizar o texto antes e depois da limpeza.

3. Preparação do Input para Normalização:

- I. Após a limpeza, o sistema deve preparar o texto para ser enviado ao modelo de linguagem. Esta preparação inclui:
 - a. Segmentação do texto em blocos adequados (chunking);
 - b. Criação automática de prompts apropriados ao tipo de normalização;
 - c. Detecção automática de idioma;
 - d. Formatação consistente (plain text).

4. Estabelecer conexão à API do SLM:

- I. Enviar um pedido HTTP POST com o texto limpo e o *prompt* a uma API que faz conexão com o SLM (<https://reality.utad.net/slm>).

Body do pedido:

```
{
  "model": "llama-3.2-1b-instruct",
  "messages": [
    {"role": "user",
     "content": "<PROMPT>"
    }
  ]
}
```

- II. Processar a resposta recebida da API.

5. Criação De Relatórios Automáticos:

- I. A aplicação deve gerar relatórios contendo:
 - a. Parâmetros na pipeline;
 - b. Texto antes/depois;
 - c. Avaliação da normalização;
- II. Os relatórios podem ser exportados em PDF ou HTML.

Regras e Avaliação

- Os trabalhos são realizados em grupo, sendo que cada grupo será constituído por 3 alunos. Somente em casos devidamente justificados poderão ser formados grupos com menos alunos, nunca ultrapassando este limite.
- A utilização de Git é obrigatória para todo o desenvolvimento do trabalho. Cada grupo deve manter um repositório Git (GitHub, GitLab ou equivalente) contendo todo o código, documentação, histórico de commits e versões da aplicação.
- A apresentação é obrigatória para todos os membros do grupo. Em caso de ausência à apresentação, sem justificação legalmente aceite, o aluno será avaliado com nota zero na totalidade do trabalho.

Nota Final

A nota final do trabalho é obtida através da utilização da seguinte fórmula:

$$NFTP = CE * 10\% + 30\% * TP + 20\% * RTP + 40\% * ATP$$

NFTP = Nota final do trabalho prático

TP = Trabalho Prático

ATP = Apresentação trabalho prático

| RTP = Relatório do trabalho prático

| CE = Cumprimento das Etapas

Datas importantes | Submissões

08/05/2026 | Etapa 01 – Pipeline de Limpeza e preparação do *prompt* (Até à 3ª tarefa).

24/05/2026 | Final – Relatório + Apresentação eletrónica + aplicação